CareerCompass AI

An AI-Powered Personalized Career and Skill Recommendation System for College Students

A Project Report

Submitted by: Yash Parmar | Milan Gagiya | Hiren Jalondhra | Vrutik Sotha

Technology Stack: Python Flask - React.js - Tailwind CSS - SQLite

Algorithm: Rule-Based Expert System - two selectable combinations: Weighted MCDA with Topological Sort, or a Decision Tree with Breadth-First Search

Report date: Oct 24, 2026

Abstract

CareerCompass AI recommends career tracks to college students and generates a complete, dated career path for each. It is built on a **rule-based expert system** that combines **Weighted Multi-Criteria Decision Analysis** for ranking careers with **prerequisite graph traversal** (Kahn's topological sort) for sequencing the skills a student must learn. Two ready-made algorithm combinations are offered and can be compared side by side: Weighted MCDA with topological sort, and a Decision Tree with breadth-first search. No model is trained, no dataset is required, and no external AI service or API key is used anywhere.

The knowledge base encodes 21 career tracks, 92 skills with difficulty, learning hours and prerequisite relationships, 242 skill aliases, and 105 role-progression stages. For any profile the engine computes an explainable 0-100 fit score for every career, identifies the exact skill gaps, pulls in missing prerequisites automatically, orders every skill so none precedes its dependencies, and converts the result into phased study weeks from the student's own study capacity. It is verified by 41 automated tests, including an exhaustive check of the prerequisite ordering and of the knowledge base's internal consistency. Because every output is deterministic arithmetic over declared rules, the same input always yields the same result and every number can be traced to a rule.

1. Introduction

Most final-year students know which degree they hold but not which role they are employable for. Job portals list vacancies without saying what is missing from the profile, aptitude quizzes return a personality label with no technical next step, and senior advice

is anecdotal.

The practical questions a student actually has are sequential: which roles am I closest to, exactly which skills am I missing, in what order should I learn them given that skills depend on one another, how long will that take at my study pace, and where does the role lead after the first job. A ranked list of job titles answers only the first. This project answers all five.

Objectives

1. Encode a knowledge base of technical career tracks covering the degree backgrounds actually present in a college, not only Computer Science.
2. Score a profile against every track with a transparent weighted formula that can be defended line by line.
3. Perform skill gap analysis that separates must-have skills from nice-to-have ones.
4. Generate a learning order that respects skill dependencies, and convert it into real weeks using the student's own study capacity.
5. Keep the whole system deterministic, offline and free of any trained model or paid API.

Why a rule-based expert system, and not machine learning

Job portals list vacancies without saying what is missing from a profile. Aptitude tests return an interest category with no technical next step. Collaborative or content-based ML recommenders need labelled data and cannot explain a recommendation or produce an ordered plan. Generic MOOC roadmaps are identical for every student and ignore what is already known.

A machine learning recommender was considered and rejected for four reasons:

1. **No credible training data.** Supervised learning needs thousands of labelled (profile, correct career) records. Fabricating them would make accuracy a measure of our own fabrication.
2. **The output is a plan, not a label.** Even a perfect classifier returning "Data Analyst" cannot say which four skills are missing, which depends on which, or how many weeks the sequence takes. That is a graph problem, not a classification problem.
3. **Explainability is required.** Every number must be defensible in a viva. Rule-based arithmetic can be traced; a neural network cannot.
4. **Knowledge was available, data was not.** Facts such as "React requires JavaScript" are known and can simply be declared. Expert systems are the textbook-correct choice in exactly that situation.

The result also runs offline, at zero cost, with no API key and no model drift between demonstrations.

2. The AI Algorithm Used

> **Rule-Based Expert System with Weighted Multi-Criteria Decision Analysis and Prerequisite Graph Traversal.**

Three classical techniques work in sequence, each solving a different sub-problem:

Technique	Sub-problem	Classical basis
Production-rule expert system	Represent domain knowledge as inspectable facts	Knowledge-based (symbolic) AI
Weighted MCDA	Rank 21 alternatives on several weighted criteria	Simple Additive Weighting
Topological sort of a DAG	Order dependent skills correctly	Kahn's algorithm, graph theory

The knowledge base decides *what the system knows*, the MCDA layer decides *which* careers to recommend, and the graph traversal decides *how* the student reaches one.

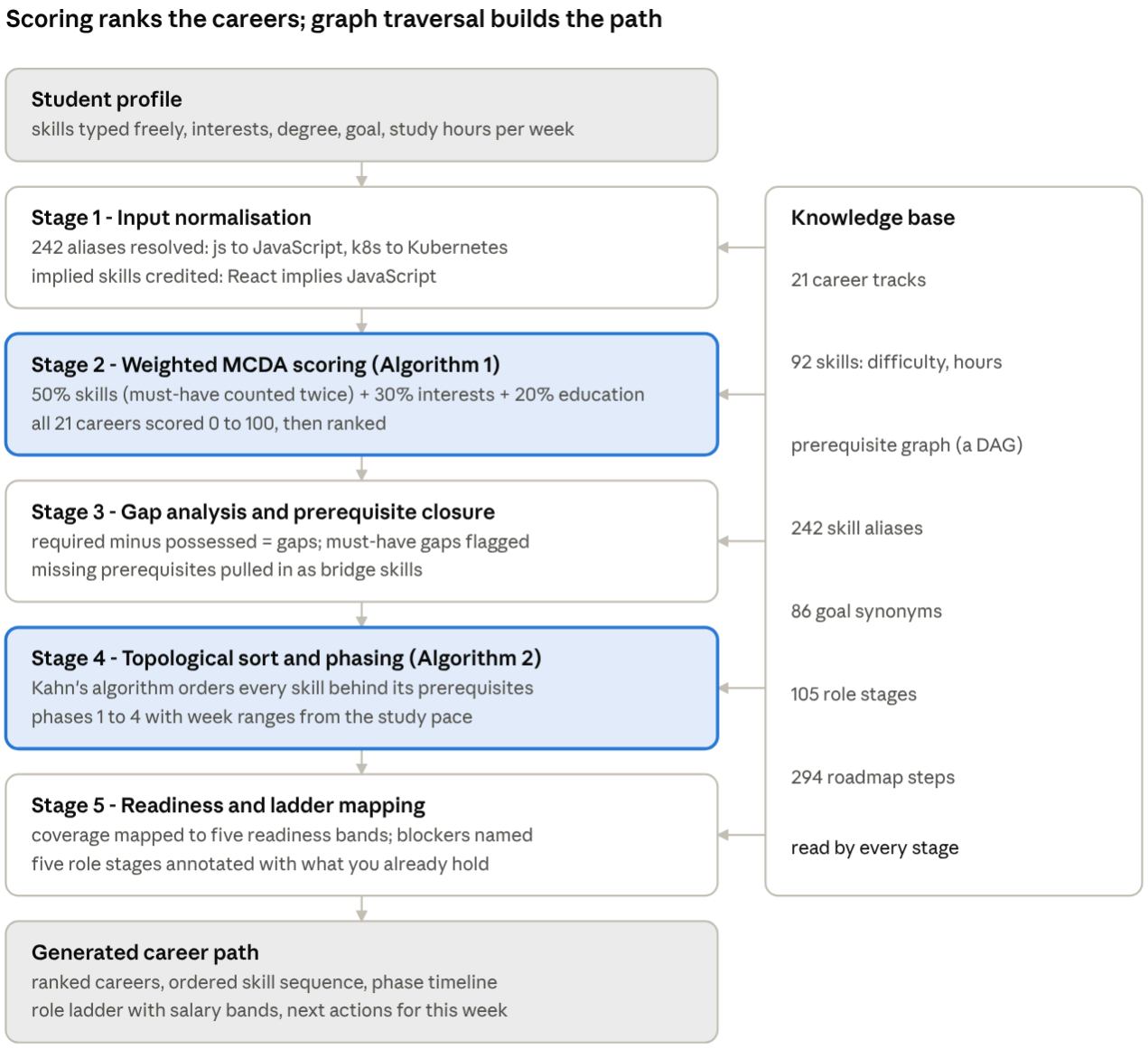
Sections 6 to 9 describe the default pair in detail. Section 13 adds a second combination, with another way to answer *which* career fits and another way to answer *how* to reach it, all selectable per request, and Section 14 compares them.

Not used anywhere: no trained model, no training dataset, no external AI service or LLM API, no API keys, and no randomness. Machine Learning appears in this project only *as subject matter*, because it is one of the 92 skills a student may be advised to learn; the recommender itself performs no learning.

Throughout this report: `R(C)` is the skills required by career `C`, `K(C)` its must-have core subset, `S(U)` the student's declared skills, `I(U)` the skills implied by them, `pre(s)` the direct prerequisites of skill `s`, `h(s)` its learning hours, `d(s)` its difficulty rank (1 to 3), and `w(s, C)` its scoring weight (2.0 if core, else 1.0).

3. The Algorithm Pipeline

A single request passes through five stages. Stages 2 and 4 are the two algorithms proper; the stages around them exist to make those two work on real student input and to turn their output into something readable.



Shaded stages are the two AI algorithms; the others normalise the input and format the result.

algorithm pipeline - 5 stages, 2 algorithms, 1 shared knowledge base

The knowledge base is read by every stage and written by none, which makes the engine stateless: two students with identical profiles receive identical paths, and a domain expert can correct the facts without touching the inference code.

4. Knowledge Representation

An expert system is only as good as its fact base. This is what the algorithm knows before any student arrives:

Element	Count	Element	Count
Career tracks	21	Skill aliases	242

Element	Count	Element	Count
Skills	92	Career-goal synonyms	86
Skill categories	17	Role progression stages	105
Interests	37	Roadmap milestones	294

Career frame. Each career declares `required_skills` (the benchmark set used for gap analysis), `core_skills` (the must-have subset, weighted 2.0 in scoring), `relevant_interests`, `education_keywords`, a five-stage `career_path` ladder, and the portfolio projects and certifications that fill the applied phase of the generated plan.

Skill frame and the prerequisite graph. Each of the 92 skills declares its `difficulty` (Beginner, Intermediate or Advanced, mapped to 1, 2, 3), its `learning_hours`, its `prerequisites`, and three to four concrete roadmap steps. The `prerequisites` field is what makes path generation possible: taken together the 92 skills form a **directed acyclic graph**, for example CSS depends on HTML, React on JavaScript, Kubernetes on Docker which depends on Linux, and MLOps on Machine Learning, Docker and CI/CD.

Acyclicity is a correctness requirement rather than an assumption: the validation routine in Section 10 rejects any self-dependency, and the traversal carries a cycle guard so corrupt data degrades gracefully instead of hanging. All of this knowledge lives in one data file, separate from the inference code, which is the defining property of an expert system: the project grew from 5 careers to 21 and from 22 skills to 92 without rewriting the engine.

5. Stage 1: Input Normalisation and Implied Skills

Alias resolution. Students do not type skills the way a catalogue spells them. A student who types `js`, `mysql` and `k8s` means JavaScript, SQL and Kubernetes; matching raw strings would count all three as gaps the student already has. The rule applies three tests in order: case-insensitive exact match against the 92 canonical names, then lookup in a table of 242 declared aliases, and otherwise the input is preserved exactly as typed rather than discarded.

Alias class	Examples	Resolves to
Abbreviations	`js`, `ml`, `dsa`, `k8s`	JavaScript, Machine Learning, Data Structures & Algorithms, Kubernetes
Vendor variants	`mysql`, `postgres`, `sqlite`	SQL

Alias class	Examples	Resolves to
Library to parent skill	`tensorflow`, `keras`, `cnn`	Deep Learning
Tooling to discipline	`burp suite`, `kali linux`, `owasp`	Ethical Hacking

The same technique maps 86 job-title synonyms such as `ML Engineer`, `SDET` and `MERN developer` onto career identifiers.

Implied-skill inference. This rule was added after testing exposed a real logical flaw. A student listed Network Security, Linux, Ethical Hacking and SIEM, and the engine reported Computer Networking as a missing skill and scheduled it in Phase 1. That is absurd: Computer Networking is a declared *prerequisite* of Network Security, so the student necessarily already knows it.

The fix traverses the prerequisite graph **upward** from everything the student claims, taking the transitive closure:

$$I(U) = \{\, p \in G \;\mid\; p \in \mathrm{pre}^{*}(s) \text{ for some } s \in S(U),\; p \notin S(U) \,\}$$

So React implies JavaScript, Kubernetes implies Docker and Linux, and Deep Learning implies Machine Learning, Python, Statistics and Mathematics. The effective skill set used for matching becomes $S(U) \cup I(U)$.

Two safeguards keep this honest: implied skills are reported separately in `implied_skills` and shown to the student with the reason, and inference runs only along declared edges, never guessing a relationship an expert has not written down.

6. Stage 2: Weighted Multi-Criteria Decision Analysis

This stage answers "which careers fit this student?" It is a Simple Additive Weighting model: three criteria are measured independently on a 0-1 scale, weighted, and added. The result is clamped to [0, 100].

$$\mathrm{Score}(U, C) = \Big(0.50 \cdot R_{\text{skill}} + 0.30 \cdot R_{\text{interest}} + 0.20 \cdot R_{\text{edu}} \Big) \times 100 \; + \; B_{\text{goal}}$$

Criterion 1, weighted skill ratio (50%). A naive ratio of matched over required skills treats every skill as equally important, so a Frontend Developer missing React would be scored like one missing Tailwind CSS. Each career therefore declares a core subset, and matches

are weighted:

```
R_{\text{skill}} = \frac{\sum_{s \in R(C) \,\cap\, (S(U) \cup I(U))} w(s, C)}
{\sum_{s \in R(C)} w(s, C)}, \qquad w(s, C) = \begin{cases} 2.0 & s \in K(C)
\\ 1.0 & \text{otherwise} \end{cases}
```

Criterion 2, interest ratio (30%). The fraction of the career's declared interests that the student matches, capped at 1. Matching is exact or substring in either direction, but only for student terms of at least four characters: without that guard a two-letter interest such as `AI` substring-matches unrelated words and inflates the score.

Criterion 3, education ratio (20%). A graded keyword heuristic, because a degree is an imperfect signal: 1.00 when the degree string contains a domain keyword the career declares, 0.75 for a general technical or graduate term (engineering, b.tech, b.sc, bca, mca, diploma), 0.40 for any other stated degree, and 0.00 when none is given. Each career declares its own keywords, which is why commerce, design and electronics degrees score 1.00 on the tracks they genuinely suit instead of being penalised for not being Computer Science.

Goal bonus. If the student's stated target resolves to this career, 5 points are added. Near-identical tracks can score within two points of each other, and when the student has explicitly named one, a small nudge keeps their intention on top. The bonus cannot promote a poorly matched career, only break a near-tie, and it is reported separately so the student sees it was applied.

Ranking. All 21 careers are scored and sorted by a three-level key: descending score, then descending weighted skill coverage, then career title. The last two keys guarantee a total order even on exact ties, which is what makes the ranking reproducible.

Worked example. A student types `html`, `css`, `js`, `github`; selects Frontend Development, Web Applications and UI/UX Design; holds a BCA; and states the goal "full stack developer". Aliases resolve to HTML, CSS, JavaScript and Git, and the goal resolves to Web Developer, whose required skills are HTML, CSS, JavaScript, React, Node.js, Git, MongoDB and REST APIs, with the first three plus Git as core.

Criterion	Calculation	Value
Skill (weighted)	4 core matched at 2.0 = 8.0, out of total weight 12.0	66.67%
Interest	3 of the career's 4 declared interests matched	75.00%
Education	"BCA" matches the declared keyword `bca`	100.00%
Goal bonus	"full stack developer" resolves to this career	+5.0

Score = (0.50 x 0.6667 + 0.30 x 0.75 + 0.20 x 1.00) x 100 + 5.0 = 75.83 + 5.0 = **80.83%**, which is exactly what the running system reports. A naive unweighted skill count would have given 4/8 = 50% instead of 66.67%, understating a student who in fact holds every must-have skill.

The 50/30/20 split is a declared design decision: skills dominate because they are the only criterion an employer tests directly, interests matter because sustained effort depends on motivation, and education weighs least because it is already fixed. The weights live in a single dictionary, so re-tuning them is a configuration change, not a code change.

7. Stage 3: Skill Gap Analysis and Prerequisite Closure

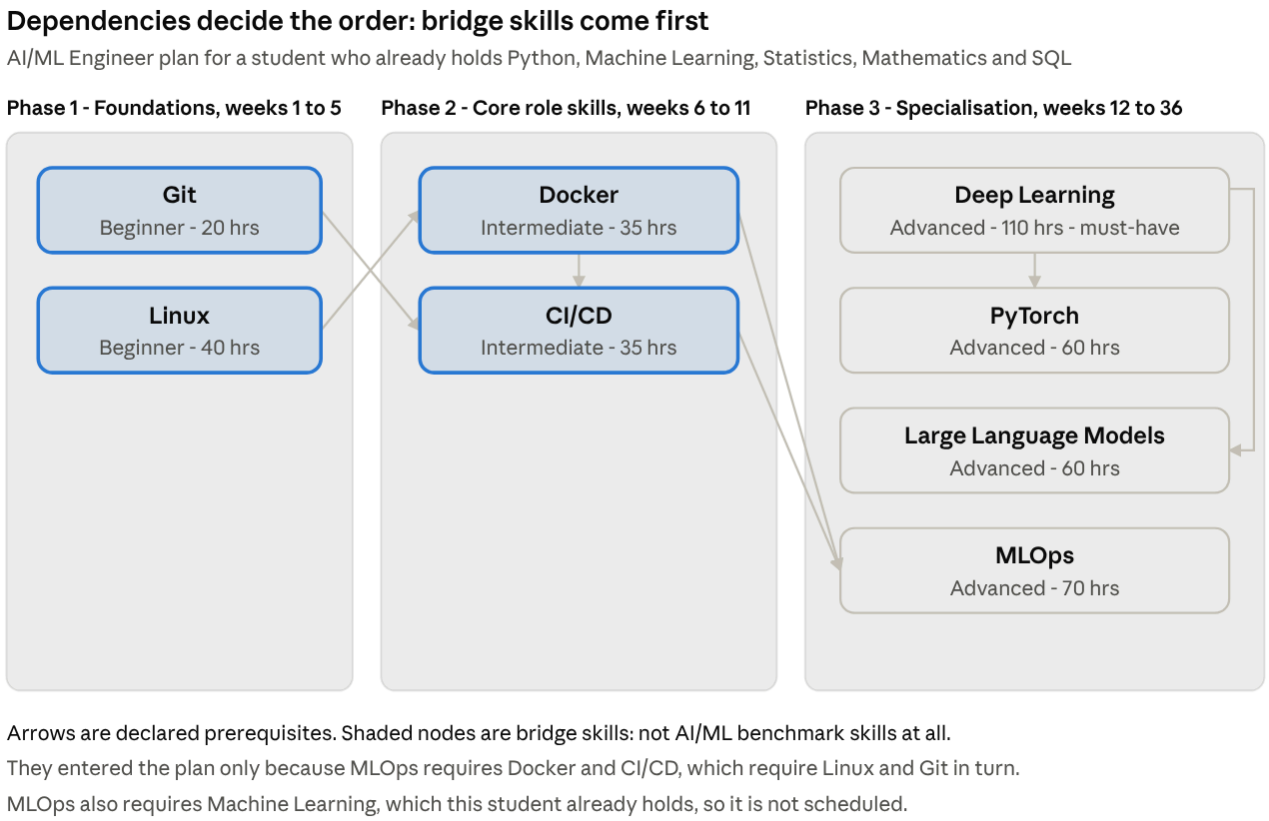
The gap is a set difference computed in the same pass as the score: `Gaps = R(C) \ (S(U) ∪ I(U))`, and the core gaps are `Gaps ∩ K(C)`. Core gaps are tracked separately because they behave differently downstream: they block the `entry_ready` verdict and are named explicitly, whereas a supporting gap is only a slower route to the same role.

A gap set alone is not a plan. For the AI/ML Engineer track, `MLOps` is required and declares `Docker` and `CI/CD` as prerequisites. Neither is an AI/ML benchmark skill, so neither appears in the gap set. Handing the student the raw gaps would tell them to begin MLOps with no container or pipeline knowledge, and nothing in the plan would ever say to acquire it.

The fix is a **downward closure** over the same graph, following prerequisite edges forward from each gap:

```
N = \mathrm{Gaps} \cup \{\, p \;\mid\; p \in \mathrm{pre}^{*}(g),\; g \in \mathrm{Gaps},\; p \notin S(U) \cup I(U) \,\}
```

Skills entering through the second term are labelled **bridge skills** and flagged in the output, so the student sees that Docker was asked of them by the dependency chain, not by the job description. The traversal is breadth-first with a visited set, giving O(m + e) time.



prerequisite closure - 4 gaps pull in 4 bridge skills across 3 phases

Four declared gaps therefore produce a needed set of eight skills. The closure is what makes the remaining stages possible: once `N` is complete and every prerequisite inside it is present, the set can be totally ordered.

8. Stages 4 and 5: Ordering, Phasing and Readiness

The closure gives the complete *set* of skills to learn. This stage decides their *order* and their *schedule*.

Why ordering needs an algorithm. Sorting by difficulty is not enough: `Microcontrollers (Arduino/ESP32)` is rated Beginner but depends on `Embedded C`, rated Intermediate, so difficulty ordering would schedule a dependent skill before its prerequisite. The order must respect dependency edges, which is what topological sorting solves.

Kahn's algorithm with deterministic tie-breaking. Each skill's prerequisite list is restricted to prerequisites that are themselves in the needed set and not already known, then every skill whose remaining prerequisites are satisfied is emitted. A plain topological sort is not unique, so ready skills are emitted by a three-level key:

```
\mathrm{key}(s) = \big(\, d(s), \; h(s), \; \mathrm{lowercase}(s) \,\big)
```

Easier first, then shorter, then alphabetical. Pedagogically this gives early quick wins; technically it makes the key a total order, so the same profile always yields the identical sequence. A cycle guard force-emits the lowest-key skill if nothing is ready, so corrupt data degrades gracefully instead of hanging.

Phase assignment. The ordered sequence is grouped into phases: Beginner skills into Phase 1 (Foundations), Intermediate into Phase 2 (Core role skills), Advanced into Phase 3 (Specialisation), with Phase 4 reserved for applied work (portfolio projects, certifications, interview preparation). The base phase is then corrected so no skill lands earlier than any of its prerequisites:

```
\varphi(s) = \max\Big( d(s), \; \max_{p \,\in\, \mathrm{pre}(s) \cap N}
\varphi(p) \Big)
```

Because the sequence is already topologically ordered, every prerequisite's phase is known by the time a skill is processed, so one linear pass suffices. The correction is visible in the example above: `Microcontrollers` has base phase 1 but its prerequisite `Embedded C` sits in phase 2, so the maximum rule pushes it to phase 2. Empty phases are dropped and survivors renumbered, so a student with no beginner gaps sees Phase 1, 2, 3 rather than Phase 2, 3, 4.

Duration. Phase hours are divided by the student's declared weekly capacity, rounded up, and phases are laid out consecutively so each gets an absolute week range:

```
\mathrm{weeks}(P) = \left\lceil \frac{\sum_{s \in P} h(s)}{H} \right\rceil,
\qquad H \in [2, 60]
```

`H` is clamped so an absurd input cannot produce a nonsensical schedule. Changing `H` scales every timeline but never alters the skill sequence or phase membership, a property asserted by an automated test.

Stage 5: readiness classification and ladder mapping

Readiness. Weighted skill coverage is mapped onto five bands by a rule that fires on the first matching threshold: 80% and above is Job Ready, 60 to 79% Interview Ready, 40 to 59% Developing, 20 to 39% Foundation Builder, and below 20% Explorer. A separate boolean adds a necessary condition:

```
\mathrm{entry\_ready} = (\mathrm{coverage} \geq 60\%) \;\wedge\; \big( K(C)
\setminus (S(U) \cup I(U)) = \varnothing \big)
```

The second clause matters: a student can reach 60% coverage on supporting skills while still missing a must-have one. That student is *not* entry ready, and the blocking core skills are named explicitly instead of being hidden inside a percentage.

Ladder mapping. The career's five static stages are annotated per student. Stage 1 is marked `entered` when `entry_ready` holds and `target` otherwise, Stage 2 is `next`, and Stages 3 to 5 are `future`. For every stage the skills it expects are split into those the student already holds and those still needed, so a ten-year trajectory reads as reachable rather than abstract.

Next actions. The three skills to start are chosen by sorting the sequence on phase number first (only skills whose prerequisites are satisfied can begin now), then core skills before supporting ones, then position in the sequence. Each is joined with its first roadmap step, and the career's first portfolio project and certification are appended. An earlier version sorted core-first before considering the phase, which told a student to begin an Intermediate core skill while two Phase 1 prerequisites were unlearned; promoting phase to the primary key fixed it. No sentence here is generated by a language model: each is composed from declared fields, which is why the output is identical on every run.

9. Pseudocode and Complexity

```
ALGORITHM RecommendCareers(U, careers, H)
   U = student profile, careers = the 21 frames, H = study hours per week

 1  S <- { CanonicalSkill(s) : s IN U.skills }         // 242 aliases
 2  E <- S UNION ImpliedSkills(S)                      // prereqs of claimed
skills
 3  FOR EACH C IN careers DO
 4      Rskill <- SUM(w(s,C) : s IN C.required AND s IN E)
                  / SUM(w(s,C) : s IN C.required)     // w = 2.0 core, else
1.0
 5      score  <- (0.50*Rskill + 0.30*Rinterest + 0.20*Redu) * 100
 6      IF GoalResolvesTo(U.goal) = C.id THEN score <- score + 5
 7      gaps <- C.required MINUS E ; N <- gaps ; queue <- COPY(gaps)
 8      WHILE queue NOT EMPTY DO                        // prerequisite
closure
 9          FOR EACH p IN Prerequisites(DEQUEUE(queue)) DO
10              IF p NOT IN E AND p NOT IN N THEN ADD p TO N ; ENQUEUE(queue,
p)
11      ordered <- EMPTY ; resolved <- EMPTY
12      WHILE N NOT EMPTY DO                            // Kahn's topological
sort
13          ready <- { s IN N : (Prerequisites(s) INTERSECT N) SUBSET OF
resolved }
14          IF ready IS EMPTY THEN ready <- { MIN of N BY SortKey }   //
cycle guard
15          FOR EACH s IN SORT(ready BY SortKey) DO     // (difficulty,
```

```
hours, name)
16              APPEND s TO ordered ; MOVE s FROM N TO resolved
17      FOR EACH s IN ordered DO                        // phases, prereq-
monotonic
18          phase[s] <- MAX(DifficultyRank(s), MAX{ phase[p] : p IN
Prerequisites(s) })
19      FOR n <- 1 TO 3 DO
20          weeks[n] <- CEILING(SUM(hours(s) : phase[s] = n) / H)
21      band <- FIRST b IN Bands WHERE Rskill*100 >= b.threshold
22      entryReady <- (Rskill*100 >= 60) AND ((gaps INTERSECT C.core) IS
EMPTY)
23      EMIT (C, score, gaps, ordered, phase, weeks, band, AnnotateLadder(C,
E))
24  RETURN results SORTED BY (-score, -Rskill, title)   // total order
```

Complexity and determinism

With `n` = 21 careers, `r` = about 8 required skills each, `m` = at most 92 needed skills and `e` their prerequisite edges:

Step	Time	Space
Normalisation, implied closure and prerequisite closure	O(m + e)	O(m)
Scoring and ranking all careers	O(n·r + n log n)	O(n)
Topological sort (naive ready-scan) and phase assignment	O(m^2) worst case	O(m + e)
Whole request	**O(n(r + m^2))**	**O(n·m)**

Because `n`, `r` and `m` are fixed by the knowledge base and not by user input, a full request runs in effectively constant time: all 21 careers scored and 21 paths generated in well under one second, with no model loading and no network call. Kahn's loop could be reduced to O(m + e) with an in-degree counter, but with `m` capped at 92 the simpler code is easier to verify and the gain is irrelevant.

Determinism. Ties in ranking and in the topological sort are broken by total-order keys, every set is sorted before it reaches output, and no clock, random seed or model sampling is read anywhere. Running the same profile twice produces byte-identical output, which can be demonstrated live.

10. Validating the Algorithm

A rule-based system has no accuracy metric, because there is no ground truth to test against. What can be proven instead is that the algorithm obeys its own rules and that the fact base it reasons over is consistent. Both are enforced by 41 automated tests.

Knowledge base self-validation. A routine cross-checks every declared fact and returns a list of problems that a test asserts is empty: core skills must be required skills, every required skill and prerequisite must name a real skill, no skill may depend on itself, aliases must be lowercase and must not shadow a real skill name, and every career must have five complete ladder stages. This caught two real defects during development: an alias shadowing a canonical skill name, and an interest used by a career but missing from the selectable vocabulary, which would have made that criterion unreachable.

Correctness tests.

Test group	Count	What it proves
Knowledge base consistency and size	5	Fact base sound: 21 careers, 92 skills, 105 stages
Ordering, closure and inference	4	Prerequisites always precede dependents; bridge skills pulled in; implied skills credited, not re-taught
Alias, scoring and ranking	10	Aliases resolve; bounds and sort order hold; the correct career ranks first
Plan structure, pace and input validation	22	Phases numbered without gaps; pace changes weeks only; malformed input rejected cleanly

The ordering proof. The key property is checked exhaustively rather than by example: the test builds a position index over the generated sequence and asserts, for every skill and every one of its prerequisites present in that sequence, that the prerequisite's index is strictly lower. The same invariant is re-verified across a full career path by the end-to-end script. All 41 tests pass and the knowledge base reports zero consistency problems.

11. Worked End-to-End Trace

One real profile through all five stages, with the values the system actually produces. **Input:** skills typed as `python`, `ml`, `stats`, `maths`, `sql`; interest Artificial Intelligence; degree M.Tech in Data Science; goal `ML Engineer`; 12 study hours per week.

Stage 1. Aliases resolve to Python, Machine Learning, Statistics, Mathematics and SQL. Upward closure adds nothing new, since Machine Learning implies Python, Statistics and Mathematics, all already claimed. The goal resolves through the synonym table to `ai_ml_engineer`.

Stage 2. Of the four core skills (Python, Machine Learning, Mathematics, Deep Learning) the student holds three, plus the supporting skills Statistics and SQL, giving a weighted coverage of 8.0 out of 13.0 = 61.54%. One interest of five matches, so `R_interest` = 20%, and the degree contains the declared keyword `data science`, so `R_edu` = 100%. Score = (0.50 x 0.6154 + 0.30 x 0.20 + 0.20 x 1.00) x 100 + 5 = **61.77%**.

Stage 3. Direct gaps are Deep Learning, PyTorch, MLOps and Large Language Models, with Deep Learning the blocking core gap. Closure adds four bridge skills: Docker, CI/CD, Linux and Git, none of which is an AI/ML benchmark skill. They enter because MLOps requires Docker and CI/CD, which require Linux and Git in turn.

Stage 4. Topological sequence: Git, Linux, Deep Learning, Docker, Large Language Models, PyTorch, CI/CD, MLOps.

Phase	Weeks	Hours	Skills
1, Foundations	1 to 5	60	Git, Linux
2, Core role skills	6 to 11	70	Docker, CI/CD
3, Specialisation	12 to 36	300	Deep Learning, Large Language Models, PyTorch, MLOps
4, Portfolio and job readiness	37 to 40	-	Projects, certifications, interview preparation

Total: 430 study hours, 40 weeks, about 9.2 months. Two rules are visible here. Git and Linux precede Docker and CI/CD, which precede MLOps, exactly as the dependency edges require. Deep Learning is emitted before Docker in the sequence yet appears in a later *phase*, because phase membership follows difficulty while the sequence follows dependency; within Phase 3, Deep Learning still precedes PyTorch, which depends on it.

Stage 5. Coverage of 61.54% falls in the Interview Ready band, but `entry_ready` is **false** because the core skill Deep Learning is missing, so the blocker is named rather than hidden inside a percentage. The ladder marks Stage 1 (ML Engineer Trainee, 6 to 15 LPA) as the target and Stage 2 (Machine Learning Engineer, 15 to 30 LPA) as next, through to Principal AI Scientist at Stage 5. The first recommended actions are to start Git and Linux, each with its first roadmap milestone attached.

12. Limitations and Future Scope

The algorithm ranks careers transparently, separates critical from optional gaps, completes unreachable plans through closure, orders learning correctly and schedules it against the student's own capacity. Its honest limitations:

1. **The knowledge base is the ceiling.** Nothing outside the 21 tracks and 92 skills can be recommended: precision inside the domain, silence outside it.
2. **Skills are self-declared and binary.** There is no proficiency level and no verification, so an over-confident student receives an over-confident score.
3. **Hours and weights are expert estimates, not measurements.** Timelines carry roughly plus or minus 30% error, and the 50/30/20 split is defensible but not empirically optimised.

Future work that would keep the system rule-based: replace binary possession with a 1-to-5 self-rating so the skill ratio becomes a weighted average of levels; verify claimed skills with a short objective quiz instead of trusting self-declaration; and compute which single next skill raises the most fit scores at once, turning the graph into an optimisation over career reachability.

13. Choosing the Algorithm

Sections 6 to 9 describe one way of reasoning. The interface offers **two ready-made combinations**, each pairing a method for ranking careers with a method for ordering the learning path. One click switches both, and the original pair stays selected, so a request that names no algorithm behaves exactly as described above.

Combination	Ranks careers with	Orders learning with	Best for
Combo 1	Weighted MCDA	Topological Sort (Kahn)	A fit score you can defend line by line, and one unambiguous next step
Combo 2	Decision Tree	Breadth-First Search	Seeing the reasoning as a path of questions, and what can be started today in parallel

Both combinations read the same normalised profile (Section 5) and the same knowledge base, so any difference in their output is a difference in reasoning, never a difference in the input they saw. Neither is trained: the decision tree is declared as data. The engine also holds a third scoring algorithm, Naive Bayes, reachable through the API and described in 13.2, which is kept out of the combinations because its scale is not comparable with the other two.

13.1 Decision Tree

A two-question classification tree, written as data so a domain expert can re-shape it without touching the traversal code. Question 1 branches on the interest group the profile points to; question 2 branches on the strongest skill family inside that branch; the leaf holds the candidate careers.

```
Q1: Which domain do your interests point to?          -> Data & AI
    Q2: Which skill family is strongest?              -> AI & Machine
Learning
        Leaf: AI/ML Engineer, Data Scientist
```

The strongest group is simply the one holding most of the student's selected interests, and the strongest family the one holding most of their skills; ties break on declared order, so the walk is reproducible. Scoring then depends on where a career was reached:

Placement	Score
At the matched leaf	55 + 45 x skill coverage
Elsewhere under the same interest branch	32 + 34 x skill coverage
Not reached by the tree	8 + 30 x skill coverage

A degree matching the track's declared keywords adds 8 points, an unrelated degree removes 8, and the goal bonus from Section 6 still applies. The output carries the decision path itself, which is what makes this algorithm readable: a counsellor's reasoning rather than a weighted sum.

13.2 Naive Bayes

Bayes' theorem applied to career suitability: instead of a score, the student gets a probability.

```
P(C \mid E) \;\propto\; P(C) \cdot \prod_{e \in E} P(e \mid C)
```

The prior `P(C)` comes from the career's declared market demand, normalised across the catalog. The evidence `E` is every skill in the career's benchmark counted as present or absent (declared likelihoods: 0.90 for a core skill held, 0.72 for a supporting one, 0.14 for a skill outside the benchmark), plus the interests, the degree and the stated goal. The product is computed in log space and normalised with log-sum-exp, so posteriors over all 21 careers sum to 100%. Laplace smoothing of 0.02 stops any single piece of evidence

zeroing a posterior, and the log-likelihood is damped by 0.35 because skills are strongly correlated and the naive independence assumption otherwise drives one career to 100% and the rest to zero.

The important behavioural difference is that **absence is evidence**: a student missing Deep Learning does not merely forfeit points on the AI/ML track, the absence actively lowers its probability, because AI/ML engineers almost always hold it. Bayes therefore answers "which career does this profile most resemble today", a different and sometimes uncomfortable question from "which career should I aim for" - which is precisely why it is kept out of the two offered combinations.

13.3 Breadth-First Search

The same prerequisite closure from Section 7, grouped by level-order traversal instead of difficulty. Level 1 holds every needed skill whose prerequisites are already satisfied; removing that level exposes the next frontier, which becomes Level 2; the process repeats until every skill has a level. A skill's level is therefore the length of its longest prerequisite chain inside the plan.

This answers a question the topological phases cannot: **what can I start today, and what does finishing it unlock?** Everything inside one level is independent, so it can be studied in parallel. The algorithm also reports the shortest prerequisite chain to the deepest skill, for example `Git -> CI/CD -> MLOps`, which explains why a skill cannot be started yet.

14. If We Choose This One, Then What?

Both combinations can be run over one profile at once, so the student sees what the other choice would have produced. Using the same student as Section 11 (M.Tech in Data Science; Python, Machine Learning, Statistics, Mathematics, SQL; goal "ML Engineer"; 12 hours a week):

	Combo 1: MCDA + Topological Sort	Combo 2: Decision Tree + BFS
Picks first	AI/ML Engineer, 61.77 fit score	AI/ML Engineer, 95.69 confidence
Second	Data Scientist, 54.62	Data Scientist, 94.15
Third	Data Analyst, 40.83	Data Analyst, 54.17
Skills to learn	8	8
Study hours	430	430
Grouped into	3 difficulty phases	4 dependency levels

	Combo 1: MCDA + Topological Sort	Combo 2: Decision Tree + BFS
Start with	Git, Linux	Git, Linux, **Deep Learning**
Duration	About 8.3 months	About 8.3 months

Both pick AI/ML Engineer first, which is itself the useful finding: the recommendation is robust and does not depend on which pair the examiner trusts. The two scores are not comparable to each other, and the report never adds them: 61.77 is a weighted fit out of 100, while 95.69 is a confidence band widened by skill coverage.

When they disagree. A student with a B.Des degree, skills in Python, SQL, Figma, HTML and Statistics, and interests in both Creative Design and Data Analysis gets two different answers: Combo 1 picks Data Scientist (44.08) while Combo 2 picks UI/UX Designer (76.50). Neither is wrong. MCDA rewards overall coverage across all three criteria; the decision tree follows the single strongest interest down one branch. A disagreement signals that the profile genuinely sits between tracks, and the student should look at both.

Same destination, different shape. Because both combinations chose the same career here, the learning content is identical: the same 8 skills and the same 430 study hours, so neither route is faster. Only the grouping changes, and that difference is not cosmetic. The topological plan defers Deep Learning to its final phase because it is rated Advanced, while BFS places it in Level 1, since its prerequisites (Machine Learning, Python, Statistics, Mathematics) are all skills this student already holds. BFS also reports the shortest chain to the deepest skill, `Git -> CI/CD -> MLOps`, which explains why MLOps cannot be started yet. The same facts, ordered by a different rule, produce genuinely different advice for Monday morning.

Which to choose.

1. **Combo 1 (Weighted MCDA + Topological Sort)** for a balanced, explainable fit score and a syllabus-style order, easiest topics first. It is the pair selected on load and the easiest to defend line by line.
2. **Combo 2 (Decision Tree + Breadth-First Search)** when the reasoning matters as much as the number, when one interest clearly dominates the profile, and when it helps to see which skills can be started today in parallel.

Naive Bayes stays available through the API for anyone who wants a probability rather than a score, and for the comparison in this report. It is deliberately not offered as a combination in the interface: its output is a share of 100% spread across 21 careers, so placing it beside a 0-100 fit score would invite exactly the false comparison this section warns against.

15. Conclusion

This project demonstrates that a well-constructed symbolic AI system can outperform a statistical one on this particular problem, not despite being rule-based but because of it. A classifier can return a job title; it cannot return a dependency-ordered, time-estimated study plan, because that output is the solution to a graph problem rather than a prediction. Pairing Weighted MCDA for ranking with topological sorting for sequencing answers both the "which" and the "how", and every number can be derived by hand.

Three design decisions proved most valuable, and each came from an observed failure: **core-skill double weighting**, because an unweighted ratio rated a student missing React the same as one missing Tailwind CSS; **implied-skill inference**, because the engine was telling a student who knew Network Security to go and learn Computer Networking; and **phase-first selection of next actions**, because core-skill-first ordering recommended an Intermediate skill while its Phase 1 prerequisites were unlearned.

None of those fixes required more data or a bigger model. Each required a clearer rule, which is the lesson of the project: where expert knowledge is available and labelled outcomes are not, intelligence is better engineered by declaring what is true than by inferring it from examples. The result covers 21 career tracks, 92 dependency-linked skills and 105 role stages, is verified by 41 automated tests including an exhaustive proof of its ordering invariant, and runs offline at zero cost with byte-identical output on every run.

References

1. Russell, S. and Norvig, P. *Artificial Intelligence: A Modern Approach*, 4th edition. Knowledge representation and rule-based inference.
2. Kahn, A. B. "Topological sorting of large networks", *Communications of the ACM*, 5(11), 1962. The sorting algorithm used in Stage 4.
3. Cormen, T., Leiserson, C., Rivest, R. and Stein, C. *Introduction to Algorithms*, 3rd edition. Chapter 22, topological sort and directed acyclic graphs.
4. Giarratano, J. and Riley, G. *Expert Systems: Principles and Programming*, 4th edition. Production rules and the separation of knowledge from inference.
5. Triantaphyllou, E. *Multi-Criteria Decision Making Methods: A Comparative Study*. Springer, 2000. Simple Additive Weighting, the basis of the scoring model.
6. Hwang, C. L. and Yoon, K. *Multiple Attribute Decision Making: Methods and Applications*. Springer, 1981. Criterion weighting and normalisation.
7. Jackson, P. *Introduction to Expert Systems*, 3rd edition. Knowledge acquisition and validation of fact bases.